

```
import pygame

from ui import UI

from world import World

from robot import Robot

from asteria_core import(
    AsteriaConfig,
    Environment,
    EnvironmentAnalyzer,
    Memory,
    MissionController,
    AsteriaBrain
)

class Simulation:

    def __init__(self):

        pygame.init()

        self.ui= UI()

        self.width = 1000

        self.height = 700

        self.screen = pygame.display.set_mode(
            (self.width, self.height)
        )
```

```
pygame.display.set_caption(  
    "Asteria Simulation"  
)
```

```
self.clock = pygame.time.Clock()
```

```
# World  
self.world = World(  
    50,  
    50,  
    32  
)
```

```
# Robot  
self.robot = Robot(  
    800,  
    800  
)
```

```
# Camera  
self.camera_x = 0  
self.camera_y = 0
```

```
self.core_robot= AsteriaConfig("Asteria",50)
self.environment=Environment()
self.environment_analyzer=EnvironmentAnalyzer(self.environment)
self.memory=Memory()
self.mission_controller=MissionController()
self.brain =
AsteriaBrain(self.environment_analyzer,self.core_robot,self.mission_controller,self.memory)
```

```
def update_camera(self):
```

```
    self.camera_x = self.robot.x - self.width // 2
    self.camera_y = self.robot.y - self.height // 2

    #max_camera_x = self.world.width * self.world.tile_size - self.width
    #max_camera_y = self.world.height * self.world.tile_size - self.height
    #self.camera_x = max(0, min(self.camera_x, max_camera_x))
    #self.camera_y = max(0, min(self.camera_y, max_camera_y))
```

```
def update_sensors(self):
```

```
    robot_x = self.robot.x
    robot_y = self.robot.y

    front = 100
    rear=100
    left=100
    right=100

    for obstacle in self.world.obstacles:
        if (obstacle.y<robot_y and abs(obstacle.x-robot_x)<40 ):
            distance = robot_y- obstacle.y
```

```

        if distance<front:

            front= distance

        if (obstacle.y>robot_y and abs(obstacle.x-robot_x)<40):

            distance=obstacle.y-robot_y

            if distance<rear:

                rear=distance

        if (obstacle.x>robot_x and abs(obstacle.y-robot_y)<40):

            distance= obstacle.x - robot_x

            if distance<right:

                right= distance

        if (obstacle.x<robot_x and abs(obstacle.y - robot_y)<40):

            distance = robot_x-obstacle.x

            if distance<left:

                left= distance

        self.environment.update_front_distance(front)

        self.environment.update_rear_distance(rear)

        self.environment.update_left_distance(left)

        self.environment.update_right_distance(right)


def run(self):


    running = True


    while running:

```

```
for event in pygame.event.get():
```

```
    if event.type == pygame.QUIT:
```

```
        running = False
```

```
keys = pygame.key.get_pressed()
```

```
self.robot.move(keys)
```

```
# Robot movement
```

```
self.update_sensors()
```

```
self.core_robot.update_distance(self.environment.front_distance)
```

```
self.core_robot.update_distance_left(self.environment.left_distance)
```

```
self.core_robot.update_distance_rear(self.environment.rear_distance)
```

```
self.core_robot.update_distance_right(self.environment.right_distance)
```

```
result = self.brain.run_cycle()
```

```
print(result)
```

```
print(
```

```
    "Sensors:",
```

```
    self.environment.front_distance,
```

```
    self.environment.left_distance,
```

```
    self.environment.right_distance,
```

```
        self.environment.rear_distance
    )

    print(
        "Direction:",
        self.core_robot.direction
    )

    self.robot.execute_command(self.core_robot.direction)
    self.core_robot.direction = self.robot.autonomous_move(self.core_robot.direction)

    # Camera follow
    self.update_camera()

    # Draw world
    self.world.draw(
        self.screen,
        self.camera_x,
        self.camera_y
    )

    # Draw robot
    self.robot.draw(
        self.screen,
```

```
        self.camera_x,  
        self.camera_y  
    )
```

```
self.ui.draw(  
    self.screen,  
    self.robot,  
    self.world,  
    result  
)
```

```
pygame.display.update()
```

```
self.clock.tick(60)
```

```
pygame.quit()
```